Tandav — Complete Project Handover

Everything a fresh conversation needs in order to continue the work

23 August 2026

Contents

1 0. How to use this document

Attach or paste this at the **start of a new chat**, before asking for anything. It is written so the reader needs no earlier context.

- Where something is marked **settled**, the decision was made deliberately after discussion. Re-opening it wastes time.
- Where something is marked **verified**, it was checked against the code or proven on real hardware. **Unverified** means nobody has run it yet.
- Repository: `D:\Projects\Tandav`, branch **`Trial`**. Everything below refers to paths inside it unless stated otherwise.

2 1. The product in one page

Tandav is a **dance studio management app**, sold to studios as a **one-time purchase**. Batches, students, attendance, fees, events and costumes, monthly progress, reports. Dark black-and-gold theme.

Local-first. Every byte of studio data lives in an on-device SQLite database and the entire app works with no internet at all. There is **no Tandav server**, no login on our side, nothing to renew, and no infrastructure anyone has to keep alive.

The shape of the problem it solves: two people in **two different physical locations** each need a **fully independent, fully offline copy** of the same studio, and the two copies must **stay in sync automatically**. One of those two people is on **Android**, the other on **iPhone**. Both devices are equal masters — neither is a client of the other.

Sync happens through a **shared Google Drive account** that the studio owns. Google is used as a dumb pigeonhole; it stores small JSON files and nothing else.

3 2. Rules that are not up for discussion

Rule	Why it exists
No recurring cost of any kind. No subscriptions, renewals, monthly or yearly charges, and no weekly/monthly/yearly re-checks or re-approvals, on either platform.	The product is sold once and owned forever. This has been settled repeatedly and must not be raised again, in any form or amount, not even as an aside.
The iPhone is always in scope.	A proposal that solves Android only is not an answer. Every plan must say what the iPhone does.
The iPhone route is a PWA — the web build, opened in Safari and added to the Home Screen.	It is the only route that carries no recurring cost and no periodic renewal. Settled.
No app stores. Android ships as a **direct release-signed APK**, handed over by hand.	Deliberate. Do not propose store distribution.
File export / import over WhatsApp is a backup, not the sync mechanism.	The user rejected manual file passing as the sync design. Drive is the transport.
Git: commit and push to `Trial`. Never `main`.	`main` holds the pre-rewrite baseline.
Line endings are settled by `.gitattributes`. Do not re-litigate them.	Already normalized once; it cost a day of noise.
Install over the top (`adb install -r`). Never uninstall to fix an install problem.	Uninstalling erases the studio's **only** copy of its data, plus its backups.
8 GB RAM machine; Gradle is capped at `-Xmx2G`.	Heavier builds thrash. Do not suggest raising it.
Answer style: short, key terms in **bold**, tables and compact bullets over long prose.	Stated preference.

Rule	Why it exists
`backend/` stays where it is. Dead code, kept as history.	Explicit instruction: leave it alone. Do not run it, migrate it, or reconnect the app to it.

4 3. Architecture

4.1 3.1 Stack

Layer	Choice
App	Flutter **3.47.1** / Dart **3.13.1**
Storage (Android)	`sqflite` — on-device SQLite, `dbVersion = 2`, **12 tables**
Storage (web)	`sqflite_common_ffi_web` — WASM SQLite persisted in **IndexedDB**; `sqlite3` pinned to **exactly 2.4.6**
State	`provider`, with a `TandavApi` facade in `core/services.dart`
Sync	`google_sign_in`, `googleapis` (Drive v3), `uuid`, `crypto`
Other	`intl`, `image_picker`, `path_provider`, `url_launcher`, `shared_preferences`
Tests	`flutter_test` + `sqflite_common_ffi` — **64 tests** in 7 files

There is **no `http` dependency at all**. `core/services.dart` kept the old remote-API method names when storage moved on-device, which is why it reads like an API client and is not one.

4.2 3.2 Layout

```
Tandav/
├── SUMMARY.md           the current overview (rewritten 2026-08-23)
├── SYNC.md              the sync design in depth — read first
├── OAUTH-SETUP.md       one-time Google Cloud Console setup
├── PWA.md               the iPhone build: flags, deploy, offline, limits
├── IPHONE-INVITE.md     the link + paste-ready message for the customer
├── ship.ps1             build + verify signature + install + capture logs
├── tools/
│   ├── verify-apk.ps1        signature gate for a hand-copied APK (no cable)
│   ├── deploy-pwa.ps1        build + prune + stamp + publish the public site
│   ├── make-web-icons.py     PWA icons from the studio logo
│   ├── fake-peer.html        impersonate a 2nd device from a browser
│   └── drive-visibility-test.html
├── mobile/                   THE APP
│   ├── lib/
│   │   ├── main.dart         RootGate → Signup / Login / HomeShell
│   │   ├── core/             services, auth_state, theme, format, whatsapp
│   │   ├── platform/         the Android/web split
│   │   ├── database/         schema, migrations, backups
│   │   ├── models/           plain data classes
│   │   ├── repositories/     8 repositories, all sqflite
│   │   ├── sync/             8 files — see SYNC.md
│   │   ├── screens/          24 screens
│   │   └── widgets/states.dart
│   ├── web/                  PWA shell: index.html, manifest, tandav_sw.js,
│   │                         icons/, sqlite3.wasm  (no sqflite_sw.js)
```

```
│   └── test/                7 files, 64 tests
├── backend/                 DEAD — kept on purpose, never built or shipped
└── scripts/e2e_smoke.py     DEAD — drives the dead API
```

4.3 3.3 `lib/platform/` — the only place that knows which platform this is

```
app_files_api.dart   the interface + UnsupportedOnThisPlatform
app_files_io.dart    Android: real paths, real files
app_files_web.dart   browser: photos and backups off, IndexedDB database
app_files.dart       picks one, and exposes platformDatabaseFactory
```

The pick is a conditional import whose **default is the web file**:

```
import 'app_files_web.dart' if (dart.library.io) 'app_files_io.dart' as impl;
```

A platform offering neither `dart:io` nor a browser therefore fails **loudly** instead of silently getting a file system it does not have.

Shared code must read `platformDatabaseFactory` from here and must never import `databaseFactory` from `package:sqflite` — that global is the Android one, and touching it from shared code is exactly what makes a file impossible to compile for the web.

4.4 3.4 Database — 12 tables

`users` · `app_settings` · `batches` · `students` · `attendance` · `monthly_attendance` · `fees` · `fee_payments` · `events` · `event_participations` · `monthly_progress` · `sync_state`

- The nine business tables carry `sync_uuid`, `device_id`, `updated_at`, `deleted_at`, added programmatically by `_addSyncColumns()` looping over `syncTables` — which is why they do not appear in the `CREATE TABLE` text.
- **`users`, `app_settings` and `sync_state` never sync.** This is a **security boundary, not tidiness**: the sync bundle is plain unencrypted JSON, `users` holds the password hash, `app_settings` holds the account recovery code.
- `students.batch_id` → `batches.id` is **SET NULL**, so deleting a batch leaves its students as "Unassigned" instead of destroying them. Student children (attendance, fees, participations, progress) **CASCADE**.
- `fees.amount_paid` is additive per recorded payment; `status` is derived (due / partial / paid). Same shape for `event_participations.costume_fee_paid`.
- Monthly aggregates are recomputed after every daily attendance save.

4.5 3.5 Two different things are called "account"

Confusing these is the most common mistake in this project.

App login — one per phone, different on each, never syncs. The `users` table is still seeded with `admin` / `admin123` on first open, because `_seedAdminIfNeeded` recreates that row whenever the table is empty. What changed is that the app now **refuses to let anyone in on it**: `isFactoryDefault()` asks whether the factory password still verifies, and while it does, an un-dismissable **signup screen** replaces the login screen. Signup updates that same row in place and issues a **recovery code, shown once**, which is the only way back in if the password is forgotten.

The check is deliberately "does the factory password still work" rather than "is the table empty" (it never is) or a separate flag (which can drift out of step with the row) — so an APK already in someone's hands gets prompted on its **next launch** instead of staying on `admin123` forever.

Google account — one, shared by both devices, mandatory. It *is* the sync mailbox. Two different Google accounts means two private Drives and sync can never work. Advise each studio to make a **fresh Google account used only for Tandav**, not either person's personal Gmail.

5 4. How sync works

Both devices sign into the same Google account with the **`drive.file` scope only**. The app creates a **`Tandav Sync`** folder, and each device owns exactly one file in it: **`tandav-<deviceId>.json`**.

A device **writes its own file and reads the other's**, so **neither waits for the other to be online** — one can sync at 9am and the other at 6pm and both end up correct. This is store-and-forward, not a connection.

Device identity is `TANDAV-XXXX`, drawn from the alphabet `23456789ABCDEFGHJKMNPQRSTUVWXYZ` (no look-alike characters), stored in `sync_state.device_id`, and generated **only when absent**.

5.1 4.1 The invariant that matters most

There are **two independent per-table marks** in `sync_state`:

Mark	Meaning
`sent.<table>`	**What the peer already holds.** The only thing gating what we upload.
`watermark.<table>`	What we have received and applied from the peer.

`sent.<table>` is a claim about one specific peer. So **anything that clears `cloud_peer_device_id` must clear the sent marks in the same transaction** — the next device adopted is a *different* device and holds nothing.

When `forgetCloudPeer()` did only half of that, the replacement device was adopted, **both sides reported a clean sync**, and the studio's entire history was never offered to it, with nothing on screen to say so. It presents as two separate faults ("the phone isn't sending", "the iPhone isn't sending") and is one. **Fixed** — and the consequence is that **"Forget the other device" now implies "Send everything again"**.

5.2 4.2 Merge rules

Last-write-wins on `updated_at`; the **higher device id breaks exact ties**; soft-delete **tombstones** rather than real deletes; foreign keys **remapped by `sync_uuid`**; the whole inbound bundle applied in **one transaction**.

5.3 4.3 When it runs

On app open, on resume, **every 5 minutes while in the foreground**, and on demand from **Settings → Device & Sync**.

5.4 4.4 Deltas, not backups

A healthy pair's mailbox files are **nearly empty**, which is correct — and it also means **the Drive account is not a copy of the studio**. A device that lost its data is rebuilt with **Send everything again** on the surviving device. The real backup path is `TandavBackups`.

Only two devices may share the account, counted as one `tandav-*.json` per device in the folder. A browser tab used for testing is a **real peer occupying a slot**.

A **Bluetooth transport existed and was deleted on 2026-08-23** (44 files, including the vendored `ble_peripheral` plugin). Drive is the only carrier. Do not resurrect it.

6 5. Distribution

6.1 5.1 Android

A direct, **release-signed** `.apk` per studio. Run **`.\tools\verify-apk.ps1`** before copying one to a phone — it needs no cable.

No store means no auto-update channel: a new release is redistributed by hand, and Play Protect will warn on install.

6.2 5.2 iPhone — the PWA

The same Flutter app compiled for the web and served over HTTPS at:

`https://jagansk06.github.io/tandav-app/`

Installed from **Safari → Share → Add to Home Screen**. Not a native app, and nothing to renew.

Hosting is GitHub Pages — settled 23 August 2026. The source repo stays **private**; the built site goes to a **second, public** repo holding **nothing but build output**, because Pages on the free plan will only serve a public repository.

	Repo	Branch	Visibility	Holds
Source	`jagansk06/Tandav`	`Trial`	private	the app
Site	`jagansk06/tandav-app`	`main`	**public**	`build/web` only

The site repo is machine-generated: every deploy **replaces its single commit with a force push**, so it never grows and nothing placed there by hand survives.

Build flags are load-bearing, all three of them:

```
flutter build web --release --pwa-strategy=none --no-web-resources-cdn --base-href /tandav-app/
```

- `--pwa-strategy=none` — on Flutter 3.47 the generated `flutter_service_worker.js` is an **815-byte tombstone whose whole body unregisters itself**. So "it's a PWA, Flutter handles offline" is **false**. The offline support is **ours**: `mobile/web/tandav_sw.js`. The flag stops Flutter registering a second worker for the same scope.
- `--no-web-resources-cdn` — otherwise CanvasKit is fetched from `gstatic.com`: cross-origin, therefore uncacheable, therefore **no offline app**.
- `--base-href /tandav-app/` — must match the served subpath. A mismatch is a blank white page with a 404 per asset, which reads like a broken build rather than a wrong string.

`deploy-pwa.ps1` also **prunes about 24 MB the browser can never request** (debug symbols, the `skwasm`/`wimp` files only a `--wasm` build loads, `webparagraph`, any stale `sqflite_sw.js`), then **stamps a content-derived cache version** into the service worker so a redeploy of an identical build lands on the same cache and customers re-download nothing.

Only `$GitHubUser` and `$SiteName` at the top of that script are meant to be edited; the base href, site URL and OAuth origin all derive from them so they cannot drift apart.

6.3 5.3 What the iPhone customer will actually notice

Two things, both **browser limits rather than choices**:

1. **Photos and Backup/Restore are hidden.** A browser has no file paths, and half-working was worse than honest.
2. **Sync needs one "Resume syncing" tap per launch.** A browser gets a short-lived access token and no refresh token, and Safari keeps it in memory only, so a reload has nothing to restore and

minting a new token needs a pop-up, which needs a tap. One tap covers roughly an hour. The Device Sync screen says “Signed in as …” rather than “Not connected” in this state, so a healthy customer is never told sync is broken.

Everything local — students, attendance, fees — needs **no tap and no signal**.

6.4 5.4 The keystore must survive forever

`D:\Projects\tandav-signing\tandav-release.jks`

Two severe reasons: a **differently-signed APK cannot update an installed one**, and the only way through is uninstalling, which **destroys the studio’s only copy of its data**; and the **Google OAuth client is bound to that keystore’s SHA-1**, so a build signed with anything else cannot reach Drive at all.

Never commit it, never regenerate it, keep it in two backup locations. `mobile/android/key.properties` holds its password and must never be committed.

7 6. Security boundaries

Item	Rule
`tandav-release.jks`, `key.properties`	Never committed, never sent over WhatsApp or email, never regenerated. Gitignored.
`device-log.txt`	Gitignored — it can contain account details.
Password hash, recovery code	**Never** enter the Drive bundle. That is why `users` and `app_settings` do not sync.
Backup files (`TandavBackups`)	A copy of the **whole `.db`**, so they contain the password hash and the recovery code in plaintext. **Treat a backup as a secret.**
OAuth **Web client ID**	**Not a secret.** Public by design, safe to commit. The **Authorized JavaScript origins** list is the actual security boundary.
Scope	`drive.file` **only**. Never `drive` — the broad scope is *restricted* and drags in a recurring third-party security assessment.

8 7. Decision log

Decision	Reason
Local-first, no server	Nothing to pay for, nothing to keep alive, works with no signal.
Google Drive as the transport	Free, already owned by the customer, and store-and-forward means neither device waits for the other.
`drive.file` scope only	Avoids restricted-scope review, and is honest: the app genuinely cannot see the customer’s other files.
Publish the OAuth app to production	While it says *Testing*, every refresh token **expires after 7 days** — sync would work for a week at each customer and then stop forever.

Decision	Reason
iPhone = PWA	The only route with no recurring cost and no periodic renewal.
Separate **public** site repo	Pages needs a public repo; the source must stay private.
GitHub Pages over the alternatives	Chosen 2026-08-23. One well-known alternative's free tier is explicitly **non-commercial** and its paid tier is a monthly fee — disqualified. Cloudflare Pages remains the fallback if ever needed; it would only mean rewriting the publish half of `deploy-pwa.ps1` and setting `--base-href /`.
Bluetooth transport deleted	Two transports meant two sets of merge bugs for no gain.
First-run signup replacing `admin123`	A shipped default password on an app holding a studio's records.
Recovery code, plaintext, never rotated, shown once	There is no server to reset a password from.
`sqlite3` pinned to 2.4.6	Must match the hard-coded `sqlite3.wasm`, or WebAssembly LinkError.
Worker-free web database factory	The shared-worker handler answers every internal failure with `postMessage(null)`, so real errors never reach the console — and iOS Safari has no `SharedWorker` anyway.
`wa.me` links on web	Safari will not hand a `whatsapp://` scheme from a web page to another app: no chat, no error.
`backend/` kept but dead	History, not a component.

9 8. Status

9.1 8.1 Working, and validated on real hardware

The full Drive round trip (OAuth, folder creation, encode, upload, list, download, decode, protocol validation, merge, peer adoption); first-run signup and recovery code; restore-from-backup without cloning the sync identity; recoverable peer id; Drive call timeouts; foreground periodic sync; and — **confirmed 2026-08-23 — two-way sync between the Android app and the web build**, each picking up the other's edits, with no duplicates and no echo.

9.2 8.2 Not done

1. **Two real Android phones side by side.** Every stage is proven, but one half was proven with a browser standing in for the second device.
2. **The PWA has never been on an iPhone.** It runs in Chrome on Windows, keeps its database in IndexedDB, signs in to Drive and syncs both ways. What remains: **deploy**, the **offline test against the served build**, and a **borrowed iPhone**.
3. **`flutter analyze` and `flutter test` have not been run since the last code change.** In particular `forgetCloudPeer()` changed signature from `Future<void>` to `Future<String?>` and **no compiler has seen it yet**.
4. **Conflict resolution under a wrong clock.** LWW compares wall-clock `updated_at` with no logical clock, so a badly-wrong-clock device still wins or loses every *conflict*. The *data-loss* half is already fixed. The proper fix is a monotonic per-row `local_seq` at schema v3, deferred because `SyncStamp` is synchronous and used by every repository write.

5. **`students.photo_url` is a local absolute path in a synced table.** Photos have therefore **never** travelled between devices: the path arrives, points at nothing, and `imageAt` falls back to the initial-letter avatar because it checks `existsSync()`. It degrades quietly, which is why it went unnoticed. Fixing it properly means moving photo **bytes** into the bundle.
6. **Backups cannot be moved off the phone yet.**
7. Cosmetic `flutter analyze` info lints. No errors, no warnings.

10 9. Traps worth knowing before changing anything

- **`flutter test` compiles only test files and what they import.** No test imports a screen, so a screen can be broken and the suite still passes. **`flutter analyze` is the only gate for UI files.**
- **`flutter test` never compiles the web code.** The suite runs on the Dart VM, so the conditional import always resolves to `app_files_io.dart` and every `kIsWeb` branch takes the false path. `app_files_web.dart` and the web halves of `whatsapp.dart` and `drive_mailbox.dart` are reached **only** by `flutter build web`, which is therefore a required gate.
- **Unit tests use `FakeMailbox`**, so no test can catch a plugin method missing on one platform. That is exactly how `canAccessScopes()` — web-only — reached a real device before failing.
- **Seeing a file at drive.google.com proves nothing** about `drive.file` visibility. The Drive website is Google's own client with full access. Use `tools/drive-visibility-test.html`.
- **Any `sync_state` key that is written once and auto-adopted needs a user-reachable reset.** On a local-first app "reinstall to fix it" is data destruction, not a workaround. This bug has been found **three separate times**.
- **Never sideload an APK you have not signature-checked.** A missing `key.properties` or keystore makes Gradle **silently fall back to debug signing** and the build still succeeds. Symptom on the phone: a brand-new `TANDAV-XXXX`, an empty studio, and `admin123` working again — all three at once, "every time I log in". That triple is **one** fault: a fresh database.
- **`flutter run -d chrome` throws away IndexedDB between runs.** It launches Chrome with a throwaway `--user-data-dir`, so the web database is new every run — new device id, empty studio, and one more orphan `tandav-*.json` left in the Drive folder each time. Storage does persist *within* a run, so a reload looks fine. Test persistence against the deployed URL in a normal browser.
- **`kIsWeb` must be imported explicitly.** `material.dart` re-exports `widgets.dart`, which re-exports `foundation.dart` with only `show Brightness, UniqueKey`. Importing material looks sufficient and is not.
- **WASM SQLite is two artefacts that must be the same version**, and neither `flutter analyze` nor `flutter build web` can tell you they are not. Symptom: a hang on the splash screen and `unsupported result null (null)`.
- **The WhatsApp in-app browser has no Add to Home Screen.** Tapping the invite link inside WhatsApp opens WhatsApp's own browser and the install is simply **not offered, with no error**. Every set of instructions must say *open it in Safari first*.
- **Anything that swaps the whole database file must be audited for the identity it drags along.** That is what made restore-from-backup clone `device_id`.
- **The Flutter SDK in this repo is Windows-only.** `flutter analyze`, `flutter test` and `flutter build` cannot be run from a Linux sandbox, and neither can PowerShell scripts. Those gates are the user's to run.

11 10. Commands

```
# gates
cd D:\Projects\Tandav\mobile
flutter pub get
flutter analyze
flutter test                # 64 tests

# Android: build + verify signature + install over the top + capture logs
cd D:\Projects\Tandav
.\ship.ps1
.\ship.ps1 -SkipBuild       # install the existing APK, fast loop
```

```
.\ship.ps1 -LogOnly          # capture logcat only

# check a hand-copied APK before sending it (no cable needed)
.\tools\verify-apk.ps1

# iPhone: build web + prune + stamp + publish the public site
.\tools\deploy-pwa.ps1
.\tools\deploy-pwa.ps1 -NoPush    # everything except the push
.\tools\deploy-pwa.ps1 -NoBuild   # re-publish the existing build\web

# git
git push origin Trial        # never main
```

`ship.ps1` refuses to continue unless the APK's SHA-1 matches the release key, then runs **`adb install -r`** and pulls a filtered logcat into `device-log.txt`.

Testing sync with only one phone: open `tools/fake-peer.html`, which plants a valid bundle as `TANDAV-WEB1` and reads back the phone's own bundle. **Clean up in this order** — remove the planted file **first**, then tap **Forget the other device** on the phone, or it stays pinned to the fake peer and will refuse the real second device.

12 11. The runbook — exact next steps, in order

1. **Push the source.** On Windows: `cd D:\Projects\Tandav` then `git push origin Trial`. The work is already committed as **`152b70d`** — "Local-first sync over Drive, first-run signup, and the iPhone (PWA) build", 101 files changed.
2. **Run the gates:** `cd mobile`, `flutter analyze`, `flutter test`. The `forgetCloudPeer()` signature change has never been compiled.
3. **Copy `D:\Projects\tandav-signing\` to two backup locations.** Not optional, and not yet done.
4. **Google Cloud Console**, three items that block sign-in from the deployed site and fail **only after** deploying:
   - Web client → **Authorized JavaScript origins** → add exactly `https://jagansk06.github.io` (an origin has **no path**).
   - Enable the **People API**. On the web, Google's sign-in returns a token but **no identity**; the plugin then asks `people/me` which email signed in, because the Drive client cannot be built without a signed-in user.
   - Add **`userinfo.email`** and **`userinfo.profile`** to the consent screen, alongside `drive.file`. Both are non-sensitive.
   - While there, confirm the app reads **In production**, not *Testing*.
5. **Create the site repo:** `github.com/new` → name it **`tandav-app`** → **Public** → no README, no `.gitignore`, no licence. Completely empty.
6. **Deploy:** `.\tools\deploy-pwa.ps1`.
7. **GitHub → `tandav-app` → Settings → Pages** → Deploy from a branch → **`main`**, folder **/ (root)**. Give it a couple of minutes.
8. **Free the device slot before handing over.** Delete every `tandav-*.json` that is not the Android phone's — including the stale `tandav-WEBTEST.json` in My Drive root and `tandav-TANDAV-WEB1.json` in `Tandav Sync` — then tap **Forget the other device** on Android (safe now: it clears the sent marks too). Otherwise the iPhone arrives as a **third** device and is refused.
9. **Send the invite** from `IPHONE-INVITE.md`. It is already written in WhatsApp's own formatting; use it rather than composing a new one.
10. **On the iPhone:** open the link in **Safari** (not WhatsApp's browser) → **Share → Add to Home Screen** → launch from the icon → sign up → screenshot the recovery code → sign in to the shared Google account.
11. **On Android: Send everything again.** Without this the iPhone only ever receives what changes from then on, because the Drive files are **deltas**.
12. **Then verify:** the iPhone in airplane mode (offline shell), a two-Android-phone test, backups exportable off the phone, and the `photo_url` fix.

13 12. Identifiers and paths

Android package	`com.tandav.tandav_mobile`
Release SHA-1	`50:9F:E4:A2:A5:2C:80:81:A2:F3:F9:BB:F4:98:35:0B:CB:98:1A:70`
OAuth Web client ID (public)	`209094615335-ii905pbco2bvtvdrl83g6fo4mc679dm5.apps.googleusercontent.com`
Scope	`https://www.googleapis.com/auth/drive.file`
Customer URL	`https://jagansk06.github.io/tandav-app/`
OAuth origin	`https://jagansk06.github.io`
Source repo / branch	`jagansk06/Tandav` / `Trial`
Site repo / branch	`jagansk06/tandav-app` / `main` (public)
Keystore	`D:\Projects\tandav-signing\tandav-release.jks`
Deploy staging dir	`D:\Projects\tandav-site` — **wiped on every deploy**, never put anything there by hand
Drive folder	`Tandav Sync`, one `tandav-<deviceId>.json` per device
Local dev origins	`http://localhost:8080`, `http://localhost:5555` (always pin the port: `flutter run -d chrome --web-port=5555`)

14 13. Which repo docs to read, in what order

1. **`SUMMARY.md`** — the current overview. Rewritten 2026-08-23; every earlier version described a FastAPI + Postgres + JWT client-server app that no longer exists.
2. **`SYNC.md`** — the sync design and the reasoning behind the two marks.
3. **`PWA.md`** — the iPhone build end to end: flags, deploy, offline cache, limits.
4. **`OAUTH-SETUP.md`** — the Google Cloud Console setup, once.
5. **`IPHONE-INVITE.md`** — what to actually send the customer.

If any other document claims Tandav talks to an HTTP API, that document is stale.